

Lemma: Phase 2 Plan and Status

What we are building, what is done, and what is left, split into work packages

Status as of 25 September 2026 · Non-chain lane · Repository 4waan/Lemma, working fork
tyler-turnpike/Lemma

At a glance	
Phase 1	Merged into 4waan/Lemma main (PRs #1 to #4): setup and CI, core schemas, evidence maths, economic gates.
Phase 2	8 pull requests (P5 to P12). All 8 are written and reviewed. P5 and P6 are open as PRs; P7 to P12 are not pushed yet.
Open issues	50 confirmed in the last review round (4 high, 22 medium, 24 low), plus 10 gaps in the database fixes and 20 known issues in bridge apply/verify. Some of the 50 are already fixed (section 5).
Next	9 work packages (section 5), ending with clean PRs on the fork that you open.

Contents

1. What Lemma is, in simple terms
2. Architecture
3. Who builds what: the two lanes and their seams
4. What is done
5. What is left: work packages WP1 to WP9
6. Branch order and how the PRs land
7. What we need from others
8. Stage gates: what each milestone unlocks
9. Working rules

1. What Lemma is, in simple terms

Lemma sells **ready-made, tested code changes to AI coding agents** such as Cursor. When an agent is asked to do a common job, for example "add payment gating to my MCP server", it can spend a lot of tokens (money) working it out from scratch. Lemma lets it check first whether a proven solution already exists, and buy it only if it is cheaper than doing the work itself.

How one purchase works

Step	What happens	Who does it
1. Preview (free)	The agent describes the repository: language, package manager, dependency versions, Node version. It never sends code.	Bridge → Server
2. Decision	Lemma checks its catalog and answers reuse (we have one that fits, here is the price), build (nothing fits yet) or decline (setup not supported).	Resolver
3. Pay	If there is an offer, the agent pays a small amount of USDC on a test blockchain (x402).	Teammate's lane
4. Apply	The bridge checks the files have not drifted, then writes all changes or none, and installs dependencies with install scripts disabled.	Bridge
5. Verify	The bridge runs the acceptance tests in a clean environment and sends back a signed adoption receipt: it worked, or it did not.	Bridge → Server

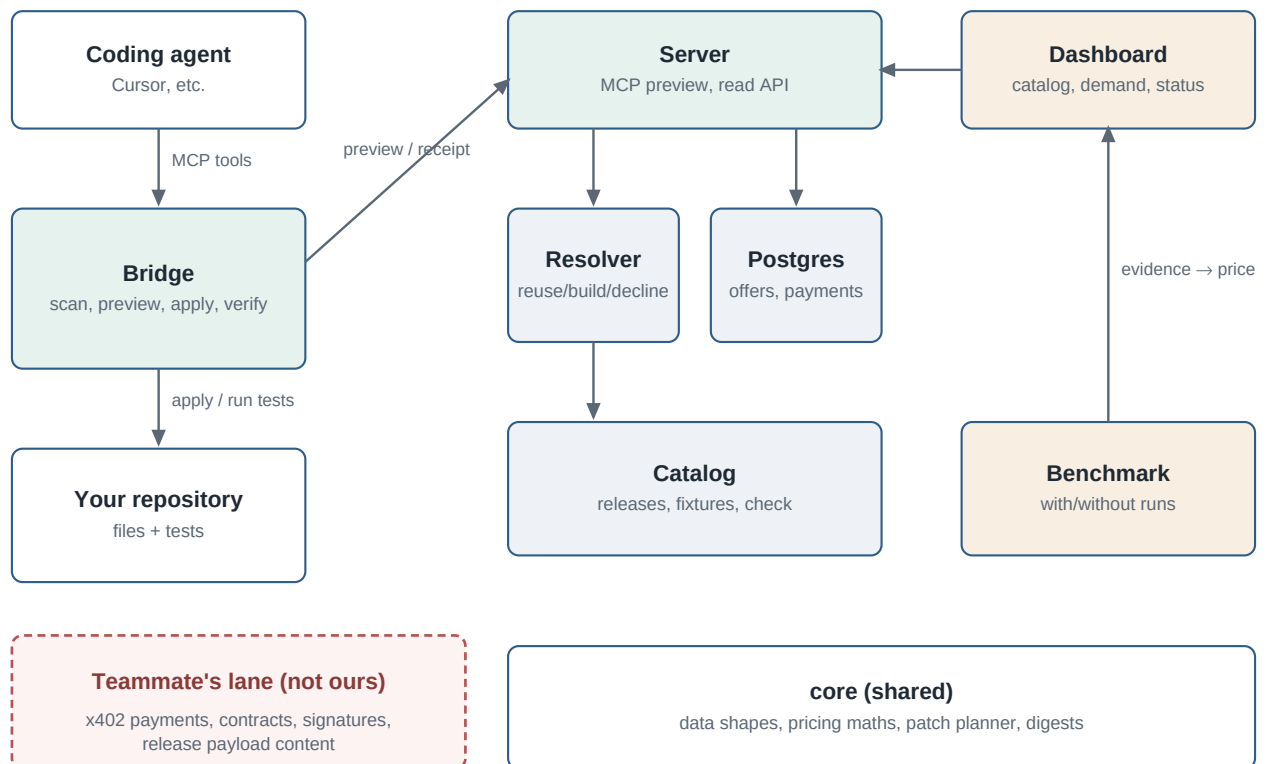
Why anyone would trust the price

A release can only be sold when a **benchmark proves it saves money**. The benchmark runs real agents on the same task with and without Lemma and measures real token cost. The saving must beat the price plus blockchain fees by a fixed margin (the 25% all-in target). The highest allowed price is computed, not guessed: `maxPriceFor(evidence, chain cost)`. The catalog checker refuses any release priced above it.

What Lemma learns over time

- **Unmet demand:** every preview is counted anonymously (only groups of at least 5 repositories are published). That gives a ranked "what to build next" list.
- **Compatibility history:** verified receipts show which releases actually work on which setups.

2. Architecture



Green: runs in production. Blue: server internals. Sand: evidence and reporting. Dashed red: the teammate's lane.

Component	Where	What it does
core	packages/core	Shared data shapes (preview, release, receipt), pricing maths (maxPriceFor), content digests, and planApply, the patch planner the bridge and the benchmark both use.
catalog	packages/catalog	The releases for sale, one test fixture per case, catalog:pack (builds bundles) and catalog:check (refuses bad digests, missing fixtures, overpricing, unbacked evidence). Also the server's startup gate.
resolver	packages/catalog	A pure function: repository profile + task → reuse, build or decline, with reasons and an offer. Deterministic, with a published ranking order.
server	apps/server	Free preview tool over stateless MCP; Postgres for offers, resolutions, receipts and anonymous demand; the ResolutionService seam for payments; read API; rate limits, CORS, security headers.
bridge	apps/bridge	Runs next to the agent. Scans package.json and lockfiles (npm, pnpm, yarn), exposes 4 short tools, predicts drift before buying, applies atomically with a journal, runs acceptance tests, and recovers a lost purchase automatically.
benchmark	packages/benchmark	Runs the with-Lemma and without-Lemma arms in isolated workspaces, reconciles real billed cost, runs the cheap stage-4 probe, and writes the evidence report.
dashboard	apps/web	Read-only web page: catalog with per-profile evidence, resolution detail, unmet demand, status. Strict CSP, no inline code.

Key design choices

- The resolver never says **adapt**; only the bridge does, when it sees the repository has drifted from what the release expects.
- The server is stateless over MCP: a new server per request, so the paid tool can be priced per request.

- A preview id is a secret between the bridge and the server; a resolution id is public. Recovery needs the preview id plus the buyer address.
- Release versions are immutable. Evidence ships as a new version X+benchmark bound to the original through baseReleaseDigest.
- Efficiency budgets are enforced by tests: Lemma's whole tool list is 3,000 characters or less, and every tool answer is 600 characters or less.

3. Who builds what: the two lanes and their seams

Our lane (non-chain)	Teammate's lane (protocol)
Catalog and resolver; free MCP tools; persistence of previews, resolutions, receipts and demand; bridge scan, tools, inbox and recovery; apply and acceptance; benchmark harness and probe; read models; dashboard.	x402 payment requirements and paid-tool wrappers; facilitator; settlement records and reconciler; bridge payment client and spend ledger; receipt and voucher signatures; smart contracts; the actual release payload code.

The seams (where the two lanes plug together)

Seam	Side	Contract
ResolutionService	Server	quote → price terms; prepare before settling (refuses double payment); commit after settling (never throws); listUnsettled / expire for the reconciler; recover for lost responses.
registerPaidTools	Server	The teammate registers the paid tool per request, with terms quoted for the preview the request names.
Inbox + paid-tool context	Bridge	Mark pending before signing, store the delivery (the bridge checks its digest), latest open offer per capability.
Signing hook	Bridge	Signs the adoption receipt; the bridge posts { receipt, previewId } to the server.
SpendLedger.entriesFor	Bridge	Amount, gas and transaction per resolution, so benchmark run records carry the real payment.

4. What is done

Phase 1 (merged)

PR	Content
#1	Project setup, CI, verify script, secret scanning (gitleaks).
#2	Core schema v1: every data shape with frozen test vectors.
#3	Benchmark evidence maths (from run records to evidence).
#4	Economic gates: the stages a release goes through before it can be sold.

Phase 2 (built and reviewed)

PR	Branch	What it adds	State
P5	core/v1.0.1-mcp-and-helpers	MCP schema fix, recovery tool name, maxPriceFor, baseReleaseDigest, planApply; lockfiles and node_modules protected from patches.	PR open. Follow-up commit ready, not pushed.
P6	catalog/loader-and-fixtures	Catalog loader, index, pack and check; two skeleton releases with full fixture sets; provisional evidence overlay (testnet only).	PR open. Follow-up commit ready, not pushed.
P7	resolver/preview-server	Resolver with golden and property tests; server with free preview over MCP, read API, rate limits, security headers.	Built, reviewed, fixed.
P8	server/persistence-and-resolution-service	Postgres (drizzle), ResolutionService, receipts, anonymous demand counting, migrations, real-Postgres race tests in CI.	Built. 14 review findings: 11 fully fixed, 3 partly. Residuals in WP1.
P9	bridge/scan-and-tools	Workspace scanner for npm, pnpm and yarn; server client; preview tool with drift probe; inbox; the Lemma rule file.	Built. Review findings in WP2.
P10	benchmark/harness-and-probe	Cursor adapter, isolated workspaces, token and cost mapping, billing reconciliation, resumable runs, the stage-4 probe.	Built. One round of fixes done; new findings in WP3.
P11	bridge/apply-and-verify	Journalled all-or-nothing apply, safe installs, acceptance runner, adoption receipts, crash recovery.	Built. 20 findings in WP6.
P12	web/read-models-dashboard	Core read models, server read routes, the dashboard (catalog, resolution, demand, status) with a strict CSP.	Built. All 14 findings fixed.

How it was checked

- Every PR ran `npm run verify` (typecheck, tests, build) and the gitleaks secret scan.
- Every branch went through a multi-reviewer audit with adversarial verification: each finding is independently reproduced before it counts.
- The last round: **50 findings confirmed, 2 refuted**. By area: bridge 16, benchmark 16, server 14 (the P8 review, largely fixed already), catalog 4.
- A dedicated check of the P8 fixes found 3 findings only partly fixed and 7 new regressions (WP1).

5. What is left: work packages

Each package lists its scope, the issues to fix, what it depends on, and when it counts as done. Severity: **H** high, **M** medium, **L** low.

WP1: Database fixes, round 2 (P8)

Scope: the gaps the fix verification found in apps/server.

Sev	Issue
M	Closing a demand day can double-count when a request arrives mid-close; a late request re-creates the day's salt, which is then never deleted. Fix: a per-day lock shared by the close and the recording; never create a salt for a closed day.
M	On a very large day (about 1M rows) the close hits the 5-second statement limit, fails every hour, and blocks later days and the purge. Fix: housekeeping gets its own time limit; each day closes separately.
M	Client addresses are stored salted with a salt kept in the same database, so they can be brute-forced. Fix: keep the daily key only in memory.
M	The paid-tool registrar is called without being awaited, so an async registrar registers too late and a store error crashes the server. Fix: make it async and awaited.
M	The receipt body changed to { receipt, previewId } but the bridge still sends the old shape, so every receipt is refused. Fix: ship the bridge change together (WP6).
L	Store errors from quote and prepare can still carry query text. Fix: wrap them as a SQLSTATE-only StoreError.
L	Connection errors are logged as just "Error". Fix: keep safe codes such as ECONNREFUSED.
L	A receipt dated more than 5 minutes ahead is refused as final, so a buyer with a fast clock loses it. Fix: a retryable answer.
L	The database URL check rejects valid multi-host and unix-socket URLs. Fix: accept any postgres:// URL and let the driver parse it.
L	The first migration was regenerated in place. Fix: document a database reset (never deployed) or add a 0001 migration.
L	Docs overclaim that queries are time-bounded; the migration-check error is reported as unreachable. Fix the wording and the error attribution.

Depends on: nothing.

Done when: all items fixed with tests, including the real-Postgres race test with rows in every trial; verify and gitLeaks pass.

WP2: Bridge scan and tools (P9)

Scope: 16 findings in apps/bridge.

Sev	Issue
H	A lost paid response can be bought again: the latest-offer lookup ignores pending purchases, and recovery runs only at startup.
M	The latest offer is still offered after the agent was told not to buy (drift likely, probe failed, or an older offer).
M	A missing or symlinked package argument silently scans the root package instead.
M	Base-probe paths from the server are not validated, so the drift check could read files outside the workspace.
M	No timeout on server requests: a stalled server hangs the preview instead of falling back to "build it yourself".
M	npm lookup walks up to the root node_modules for a package the lockfile does not describe.
M	pnpm 7/8 single-project lockfiles have no importers block, so every dependency is unresolved.
M	Aliased, git, tarball, file: and workspace: dependencies get a fake registry version.
M	A symlinked or missing package directory is silently replaced by a parent package.
M	An .nvmrc naming an LTS codename is treated as unpinned.

Sev	Issue
L	6 more: sleep-unaware offer expiry clock, trace gaps, an empty state directory putting the inbox inside the workspace, an over-strict interest schema, symlinked lockfiles skipped not refused, fixed lockfile precedence.

Depends on: nothing (fixes are made on P9, then carried up by WP5).

Done when: all fixed with fixture tests; budget tests (3,000 and 600 characters) still pass.

WP3: Benchmark harness (P10)

Scope: 16 findings in packages/benchmark, plus yarn classic running install scripts.

Sev	Issue
M	An interrupted or crashed run leaves no trace, so a resume re-runs the slot (re-rolls the result) and its cost is lost. Two findings; fix: record the attempt before the run starts.
M	If the SDK child crashes, the agent's shells keep running into the next run.
M	The credential check reads only the harness's own environment; parent processes can still expose the key.
M	The secret-variable list misses vault tokens, RPC URLs with keys, mnemonics and PGPASSWORD.
M	freeze silently overwrites an existing version.
M	A run whose billing is late can be recorded at \$0.
M	report can pass while frozen tasks or repetitions are missing.
M	yarn classic installs are not run with scripts disabled.
L	8 more: freeze smoke run skips workspace checks, placeholder economics saved as a verdict, misleading child errors, false-positive secret names, no lock on the run log, reconcile stopping at the first failure, evidence naming the wrong release, dispose errors discarding a billed attempt.

Depends on: nothing.

Done when: all fixed with fake-adaptor tests; no real Cursor run is needed.

WP4: Catalog follow-up (P6)

Scope: 4 low findings in packages/catalog.

Sev	Issue
L	The UTF-8 scan skips all of payload/, not just payload/base/.
L	A link inside payload/ is still reported twice.
L	The no-release rule fires on every fixture when a release fails to load.
L	Several non-UTF-8 names in one directory are merged into one problem.

Depends on: nothing.

Done when: fixed as a second follow-up commit on the P6 branch; catalog:check still passes.

WP5: Restack the branch chain

Scope: rebuild every later branch on the updated earlier ones: P6 on P5, P7 on P6, P8 (with fixes) on P7, P9 on P8, P11 on P9, P12 on P8, P10 on P6. Resolve conflicts in the server's app, main and persistence files.

Depends on: WP1 to WP4.

Done when: every branch passes verify on top of its new base.

WP6: Bridge apply and verify (P11)

Scope: 20 known findings in apps/bridge.

Sev	Issue
-	Concurrency and crashes: lock the journal (owner pid and start time), skip live journals, refuse a concurrent apply, kill the install's process group on exit and on recovery, per-journal error handling at startup, fsync for durability.
-	Safe rollback: restore a file only if it still holds what we wrote; save conflicts to a recovered folder; back up every lockfile present.
-	Clean installs: an environment allowlist (no keys or secrets), yarn berry skip-build mode, pnpm ignore-pnpmfile, npm save-exact for exact versions.
-	Right target: remember the package directory per preview; refuse a package without package.json; fail instead of falling back when a newer release cannot be read.
-	Honest receipts: a receipt only after an applied (or adapted) change; no receipt when the test cannot start; package manager on the minimal PATH; retry on NOT_SETTLED or MISMATCH; keep an unsigned receipt if signing fails; post { receipt, previewId }.
-	Offline mode and output: bring loopback up and refuse clearly where unsupported; never echo arbitrary command text; honest docs about key exposure.

Depends on: WP5 (and WP1's receipt format).

Done when: all fixed with tests; an end-to-end test posts a receipt through the real bridge client to the real server.

WP7: Final adversarial review

Scope: one more independent review of the WP1 and WP6 fix deltas, the riskiest code (money state and file writes).

Depends on: WP1 and WP6.

Done when: no confirmed high or medium findings remain.

WP8: Packaging and pull requests

Scope: squash each branch to one clean commit (no trailers); gitleaks; push P5 and P6 follow-ups and P7 to P12 to tyler-turnpike/Lemma; write PR descriptions for P7 to P12, update P5, P6 and P8; finish the teammate handoff note; give you compare links.

Depends on: WP7.

Done when: every branch is on the fork with a ready description, and you open the PRs.

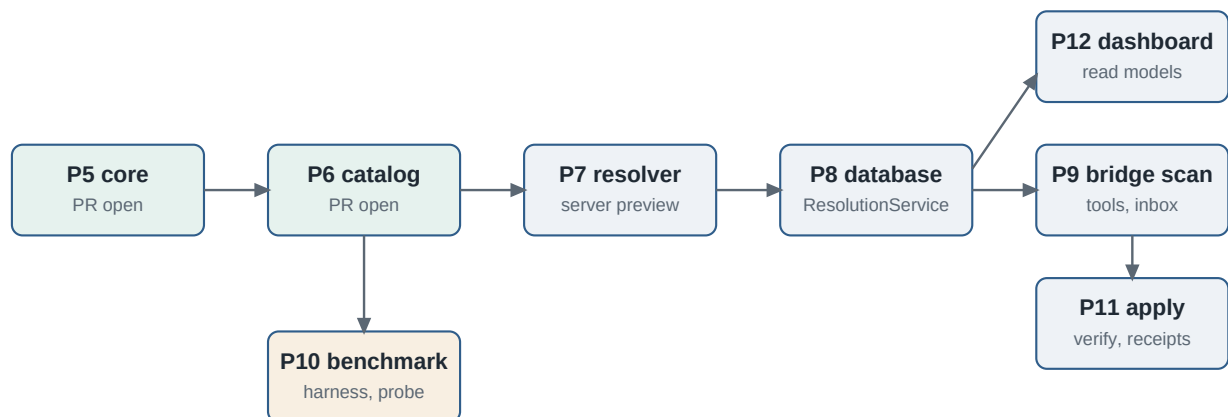
WP9: Cleanup

Scope: remove the local worktrees, stop the test Postgres, drop the test databases.

Depends on: WP8.

Done when: nothing is left running.

6. Branch order and how the PRs land



Each PR is stacked on the one before it. Reviewers review them in order; each later PR's own change is its last commit. P10 depends only on P5 and P6, so it can be reviewed in parallel with P7 to P9.

Order	PR	Notes
1	P5, P6	Already open; push their follow-up commits first.
2	P7	Needs owner sign-off on the ranking order (lever 8).
3	P8	Needs the teammate to know the ResolutionService contract.
4	P9, P12, P10	Independent of each other.
5	P11	Last; it depends on the new receipt format.

7. What we need from others

From the teammate (protocol lane)

#	Ask
1	Answer every prepare refusal with an error result, and pass the verified payer, nonce and deadline to prepare.
2	Never throw after settlement; call commit there.
3	A reconciler over listUnsettled that checks the chain and calls expire or commit.
4	Derive the payment nonce from the resolution id and the preview id; never use the resolution id itself (it is public).
5	Build the payment terms per request from quote, and return paid results as JSON text.
6	Verify receipt signatures and mark them verified (needs a small markReceiptVerified store call).
7	Export SpendLedger.entriesFor(resolutionId) for benchmark records.
8	Draft bundles for the two skeleton releases, so the stage-4 probe can give a go or kill verdict.
9	Keep the buyer's private key out of the bridge's environment (child processes can read it).
10	Fill economics.json (chain cost, price floor, ETH/USD). Until then nothing can be sold.

From the repository owner (sign-offs)

- The testnet-only provisional evidence overlay, which amends releases/README.md (raised in the P6 PR).
- The new ranking order for matching releases, lever 8 in economic-gates.md (raised in the P7 PR).

8. Stage gates: what each milestone unlocks

Stage	Meaning	Needs
2	The catalog exists and checks itself	P6
3	The free preview works end to end	P7 (P9 makes it usable from an agent)
4	Probe: is a release worth building? (go or kill, with a price)	P5, P10, a draft bundle from the teammate
5	A real paid purchase on testnet	P8, P9, the provisional overlay, the teammate's paid tool
6	Frozen benchmark with real purchases (20 runs)	Stage 5, P10, P11, the spend-ledger export
7	Deployed	All of the above

9. Working rules

- We do no blockchain, smart contract or x402 work; that is the teammate's area.
- Branches are pushed only to the fork tyler-turnpike/Lemma, never to 4waan/Lemma; you open the PRs.
- Commits and PR descriptions carry no trailers or generated-by footers.
- No PR is rushed: each is audited, with findings independently reproduced, before it is pushed.
- The core schemas are not re-audited again.
- Secrets are never pasted or committed; gitleaks runs on every commit.